

GPU Keystroke Logging and Detection on Microsoft Windows

With Kernel Mode Drivers

By Robert Strickland
University of New Orleans
November 2015

Abstract

Computers with high-end video cards are currently vulnerable to malware which can take advantage of features present on that video card to hide its activities there out of sight of the CPU and present anti-malware software.

The Direct Memory Access (DMA) functionality that video cards have allows them direct access to system memory without involving the Central Processing Unit (CPU). While this facility provides large performance benefits by allowing the video card Graphics Processing Unit (GPU) to do its massive parallel processing without slowing down (or being slowed down by) the CPU, it also provides a place for malware to snoop on system memory hidden from current anti-malware software.

This project demonstrates how a particularly valuable memory address, the keystroke buffer, can be discovered and delivered to a program running on the Graphics Processing Unit (GPU) from a malicious kernel mode driver in Microsoft Windows without doing memory scans. The program uses Direct Memory Access (DMA) to view the buffer and records the keystrokes into video card's memory for a time before encrypting and transmitting them out to another PC.

The second part of this project demonstrates how CPU-bound processes can use the Peripheral Component Interconnect (PCI) bus to examine video card memory, exposing the activities of any malware which might be hiding there. Some potential performance issues were discovered and simple ways of mitigating them were found.

Contents

Abstract.....	1
Introduction.....	3
Test Hardware Setup.....	4
Exploit Structure.....	4
KMDF Driver.....	4
CUDA Program.....	4
Listening Program.....	5
The KMDF Driver.....	5
The Keystroke Buffer.....	5
Extracting the Buffer.....	6
Sending the Offset.....	6
Rewriting the Page Table.....	6
32-Bit Addressing.....	7
Physical Address Extension.....	7
Large Pages.....	7
Flushing the Translation Lookahead Buffer.....	7
Updating the PFN Database.....	7
Keystroke Logging.....	8
Buffers and Pointers.....	8
Logging and Transmitting.....	8
Difficulties.....	9
Performance.....	9
Detection.....	10
NVIDIA Memory Dump.....	10
PCI Address Space.....	10
PCI Video Memory Dump Driver.....	10
BAR 0.....	11
Control Device.....	11
Dump Client.....	11
Proof of Concept.....	11
Performance.....	12
NovaBench.....	12
Baseline Test.....	12
No Pause Write.....	12
No Pause No Write.....	13
10 Second Pause.....	13
60 Second Pause.....	13
Performance Conclusions	14
Speculation and Future Work.....	15
Illustrations.....	16
Tables.....	16
References.....	17

Introduction

While video cards were once used exclusively for the rendering of video data, the increase in specialized computational power built into them for graphics processing has made many of them capable of performing general purpose computing tasks. This rise of General Purpose computing on Graphics Processing Units (GPGPU) has allowed the exploitation of the massive parallel computing abilities of Graphics Processing Units (GPUs) to significantly improve the computational performance for solving certain types of problems. This new computing resource is a potential windfall for the authors of malware due to design decisions made in the interest of performance with inadequate security planning. The design decisions at issue here are:

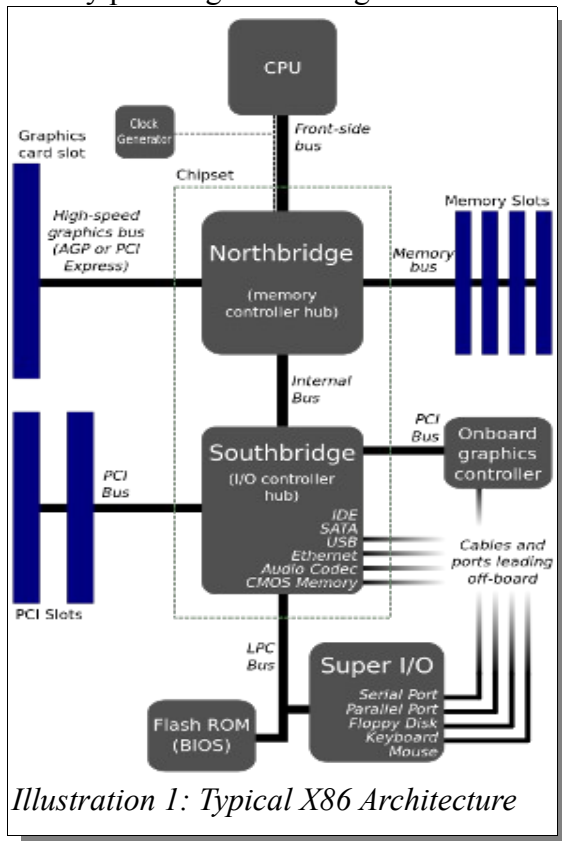


Illustration 1: Typical X86 Architecture

- Direct Memory Access (DMA) that allows the video card unmonitored access to the host computer's memory. This is supported in the hardware bus, where the Northbridge chip provides direct access to the system RAM to the GPU without routing it through the CPU. In bypassing the CPU significant performance increases can be realized¹. Although some of the risk posed is mitigated by DMA requiring a valid reference to the memory address before granting access, a system which been compromised into handing over a sensitive memory address will no longer have any means for knowing what the program is doing with that memory address².
- The inclusion of large amounts of memory on the video card which can be accessed only by the video card. Data structures allocated by a GPU process and held on the video card are not viewable by processes running on the CPU. Without access to the data structures, heuristic detection of malware is made much more difficult.

It does not matter if the video card is even being used for GPGPU purposes, the very presence of a high-end video card is what provides the resources for evading detection to an attacker. The hardware provides a blind spot into which the CPU cannot see, and over which the Operating System has little control. This paper describes a keyboard logger implemented to exploit this blind spot, and some possible ways of gaining a window to view into video card memory for detecting exploits and aims at ultimately assisting in the eventual removal of this blind spot³.

¹ [Wilt] p. 79-80

² This does not mean that DMA is undetectable by the CPU, it has been demonstrated that there are detectable side-effects that can be measured under the proper circumstances. [Stewin] p. 5-6

³ In addition to providing a blind spot in which to operate an exploit, the massive parallel computation capabilities of the many core GPU provide the ability for malware to use more sophisticated armoring techniques for the initial infestation. "GPU-based unpacking and runtime polymorphism" have been demonstrated as feasible techniques to frustrate "existing malware detection and analysis systems." [Vasiliadis] p.1

Test Hardware Setup

The techniques used, especially for the PCI Video Memory Dump, are largely hardware dependent. The HOST_MEM register and the PRAMIN window are also unavailable on cards older than the Nvidia GeForce 8 series⁴. It is also possible that the relative memory addresses may be different depending on the card used. The test hardware used for this work was a Windows 7 32-bit PC with:

- AMD A6-6400K APU
- Gigabyte F2A58M-HD2 Motherboard
- 4 GB RAM
- A PNY GeForce GTX-750 video card with 1024 MB DDR-5 on-board RAM
- A TP-LINK Wireless USB adapter
- A Lenovo ThinkPad Compact USB keyboard

Exploit Structure

The project is broken into 3 main parts, a Kernel Mode Driver Framework (KMDF) driver which finds and extracts the keystroke buffer, a user-mode CUDA program which obtains and then watches the extracted buffer periodically transmitting it, and a Listener program that runs on a remote machine to receive the buffer sent by the CUDA program.

KMDF Driver

The Driver does the work that can only be done inside the kernel – dealing with the USB keyboard device and its related keystroke buffer. The Driver is split into 3 parts:

- The Driver Entry piece which finds the Keystroke Buffer and sets up a Control Device
- The Control Device which listens for input and a memory pointer from the CUDA program and sends it to be remapped by the Page Table Manipulation piece
- The Page Table Manipulation part which takes two memory addresses supplied to it by the Control Device and alters the page table entry of the one supplied by the CUDA program to point to the address of the Keystroke Buffer.

CUDA Program

The CUDA program runs in user-mode and does the actual logging of the keystrokes. The CUDA program operates by doing the following (note that the first 2 steps may no longer be necessary, but they have not yet been fully evaluated and removed):

- Discovering the offset of the Keystroke Buffer from the KMDF Driver
- Constructing a pointer with a matching offset
- Passing this pointer to the KMDF Driver so it can be remapped to the Keystroke Buffer
- Launching a method to be executed by the GPU which reads this buffer (via DMA) saving it in a buffer in video memory.
- When the buffer is full, the buffer is encrypted and copied to a host memory address which is

⁴ The Nvidia GeForce 8 series (2006) were the first cards with the NV50 core chipset [Wikipedia], which was the first one to have both PRAMIN access at 0x700000 and the HOST_MEM window control at 0x1700 [freedesktop]. Earlier chipsets (NV40 and NV03) had some, but not all of the attributes necessary [ibid.]

passed to a spawned thread which transmits the data to the remote listening program

Listening Program

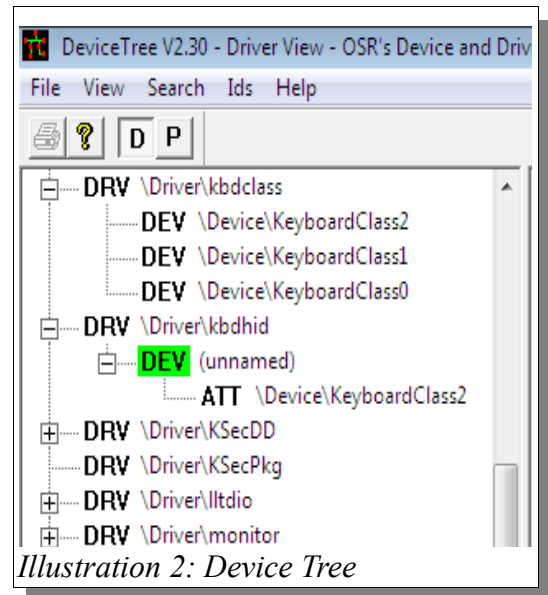
The listening program simply sits on a remote computer listening for data streams. When it gets one it decrypts it and displays it on the screen.

The KMDF Driver

The Keystroke Buffer

In the following example (see illustration 2), the USB keyboard is represented by `\Device\KeyboardClass2`. There was also a PS/2 keyboard represented with `\Device\KeyboardClass0`.

A method used in constructing keyloggers is to use `IoAttachDevice` on the keyboard to attach it to a struct so that the IRP stack can be hooked. Attempting to attach a USB keyboard will result in a `STATUS_SHARING_VIOLATION` because `\Driver\kbdhid` enforces the file sharing semantics for the keyboard device⁵. These semantics are not enforced by `\Driver\kbdclass` which is why the technique works for PS/2 keyboards. This security measure can be sidestepped, however, on a Windows computer with both a PS/2 and a USB keyboard. This is done by first calling `IoAttachDevice` to attach the PS/2 keyboard, `\Device\KeyboardClass0`. The driver object, `\Driver\kbdclass`, can be obtained from the device, and from that driver, all attached devices can be obtained, including `\Device\KeyboardClass2`, the USB keyboard.



Once the keyboard device is attached to the driver, the IRP stack is hooked, and the data structure from the IRP stack can be analyzed and output to the debug window, correctly revealing keystrokes as they were pressed. The obvious problem with this approach is that it only works in rare cases where the user has both a USB and a PS/2 keyboard connected to his machine.

Another means of obtaining `\Driver\kbdclass` is to use an unpublished API call, `ObReferenceObjectByName` which allows `\Driver\kbdclass` to be directly obtained without first going through an attached device object. Once this driver object is obtained, all attached devices, including the USB keyboard can be seen and attached.

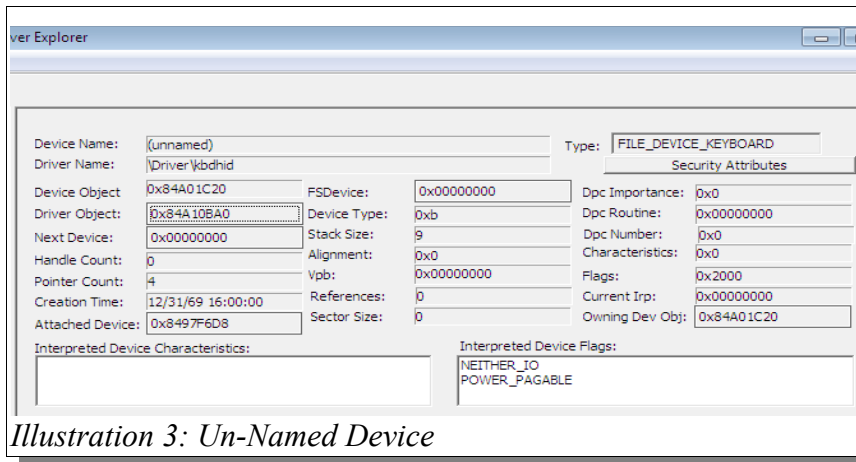
Once a pointer for the keystroke buffer was found and extracted from the kernel into a user mode process, the value of the key pressed was no longer present. Key-up and Key-down events were visible, but the value of the key itself was no longer visible. Further testing demonstrated that every so often the value of the key pressed became visible, but most of the time it was not. It appeared that the memory address would contain the keystroke for a brief instant, probably just until the IRP had

⁵ [Rossovich]

completed passing through the attached drivers, and then it was cleared.

The unnamed device between `\Driver\kbdhid` and `\Device\KeyboardClass2` (see illustrations 2 and 3) was found to contain a memory location that held and kept the most recent keystroke value. The buffer contained a value representing the last key pressed which remained until

another key was pressed. This meant that there was no way to determine how many times in a row the same key had been pressed – but it also meant that the keystroke was not as likely to disappear before being read. In order to detect multiple keystrokes and to know how many times the key was pressed, the make code (i.e. key up/key down) was also needed and this was not stored nearby.



The make code was reliably visible in the address close to the IRP, so for this implementation, 2 buffer pointers needed to be extracted: the one identifying the key that was either pressed or released, and the one identifying which key had been pressed or released.

Extracting the Buffer

Extracting the buffer from the Kernel requires getting a pointer from a user-mode program and altering the page table to point to the memory address of the buffer. The implementation required that the offset portion of the user-mode memory address match the kernel mode memory address offset.

The central aspects of the buffer extraction are:

- Sending the offset
- Rewrite the PTE
- Flush the TLB
- Update the PFN database

Sending the Offset

The offset is the last 12 or 21 bits⁶ from the virtual address of the kernel address. This is sent to the client so that the client can construct a pointer with a matching offset. This allows the page in the PTE for the client pointer to simply be replaced with the page referenced by the kernel pointer PTE.

Rewriting the Page Table

There are 3 different memory addressing schemes that could be encountered. Only 2 are dealt with here: regular 32-Bit addressing and 32 bit Physical Address Extension (PAE). With either addressing scheme, the last 12 bits (for small pages) of the address are the offset within the page.

⁶ Last 12 bits for a regular page, last 21 bits for a large page

32-Bit Addressing

In regular 32-bit addressing the other 20 bits are split into a 10 bit Page Directory Entry and a 10 bit Page Table Entry⁷. To find the Page Directory Entry one begins at the base address for the Page Directory (0xC0300000) and adds the product of the Entry # with the Page Directory Entry size (4 bytes.) This address gives a location to the Page Table Entry for the address.

Physical Address Extension

In PAE addressing the remaining 20 bits are split into 3 pieces. The first 2 are the Page Directory Pointer Index, the next 9 are the Page Directory Entry, and the last 9 are the Page Table Entry⁸. This is significant when traversing these tables in physical memory because the Page Directory Pointer indicates which (of 4) page directories to use, and the rest goes on from there. The way this is implemented, the Page Directories are stacked back to back in virtual space. What this means is that when operating with virtual addresses, one can simply use the left 11 bits as if they were a Page Directory Entry and then the rightmost 9 bits as the Page Table entry. Additionally the size of each table entry (PDE and PTE) is 8 bytes, which, in turn, requires the Page Directory to begin at a later address (0xC0600000).

Large Pages

The other issue that came up during development was that on some machines the buffer was located in a large page. In this case there was no kernel PTE entry to remap, and performing a rewrite and remap on the user-mode PDE was problematic for a number of reasons. To take a small page PDE, rewrite it and set the Large Page flag caused other faults elsewhere and resulted in blue screens warning about corrupted page tables. Allocating a Large Page from user mode appears to have been made deliberately difficult to implement. It requires the user account running the program to currently have the SeLockMemoryPrivilege privilege which has to be specifically requested even for a user who is allowed to have it⁹. Doing all these security calls raises the profile of the program which is undesirable.

This was solved by using a regular memory address, but one where the rightmost 21 bits of the address matched the rightmost 21 bits of the Kernel pointer address. The PFN value to use for the user-mode PTE was calculated based on the actual physical address of the buffer and the offset.

Flushing the Translation Lookahead Buffer

This is the only part of the process which had to be done in assembly. Since only 32-bit versions of Windows were being targeted, this could be done with 3 lines of assembly in-line. A 64-bit version should not be difficult to accommodate.

Updating the PFN Database

Windows keeps track of how many pointers reference a given address in the PFN Database¹⁰. When a new pointer references a PFN, the counter is incremented, and when that pointer is dereferenced, the number is decremented. When the counter reached 0, that memory is made available for reuse.

7 [Russinovich] pp 762 – 764

8 [Russinovich] pp 769 – 771

9 This is because large pages are non-pageable [Chen]

10 [Russinovich] pp 804 – 806

This “Database” is just a collection of bytes of a fixed size, that is, one collection for every PFN stacked in order on top of each other. For a given PFN, the address of its Database Entry can be found by going to the base address (0x83A00000 in the case of my “victim” machine) and adding that address to the product of the PFN and the PFN entry size (0x18 for a 32 bit machine, and 0x1C for a PAE machine.) The problem of automating the discovery of the PFN Database base address was not addressed – the address is subject to change with each OS release, and potentially each service pack¹¹.

By rewriting the PFN database, the computer is prevented from getting a blue screen after the CUDA program exits, but this only works the first time. If the buffers are extracted a second time, the computer will blue screen with an error about a corrupt PFN Database once that program exits.

Keystroke Logging

The logging of keystrokes is done by the GPU so as to avoid traditional detection methods. From the point of view of the user-mode CUDA program the extraction process involves:

- Getting the offsets for the buffer values.
- Creating pointers with those offsets.
- Sending those pointers to the KMDF driver to be re-written.
- Mapping those pointers to the GPU.
- Reading them for a while and storing them in an internal buffer until the buffer fills.
- Encrypting the buffer in the GPU while copying it back to a user-space address.
- Transmitting the encrypted buffer to a remote computer via a spawned thread while the program goes back to logging more keystrokes in the GPU.

Buffers and Pointers

The CUDA program looks for a handle created by the KMDF driver. If that handle is found, it transmits a DeviceIoControl with an instruction requesting the offset for the keystroke buffer. Once that is received, it then sends a second request for the offset for the make code buffer.

Once both offsets are obtained, the program begins allocating pointers until two pointers are found which satisfy both offsets. The remaining pointers are all freed.

The pointers are then sent, one at a time, to the KMDF driver to be remapped to the keystroke and make code buffers. Once the pointers are remapped, they are returned to the CUDA program which registers them to be mapped and gets device pointers for them.

Logging and Transmitting

The process for logging and transmitting the keystrokes is to call a CUDA routine to look at the buffers and to append any keystrokes found to a keystroke log buffer hidden in the video device memory. Once the keystroke log buffer is full, another CUDA routine is called to encrypt and copy that buffer into a host mapped variable which is then sent to a spawned thread for transmission to a remote computer.

¹¹ There were multiple references to this on various discussion forums, but nothing authoritative on the point. During testing the original PFN Database base address was reliably used up until the point when some Windows updates were applied to the operating system, at which point the KMDF driver behaved as if the PFN Database base address was incorrect.

The encryption takes place at the point of copying to take full advantage of the parallel abilities of GPU computing and to avoid potentially slowing down the capture of keystrokes. While the rather trivial encryption technique used in this example could have easily been done with minimal risk of missing keystrokes, a more sophisticated encryption technique might slow down the logging enough to miss some keys.

Keeping the video memory key log buffer in plain text also allows for easier analysis with our currently rather limited toolset. One may assume that an actual attacker might encrypt on the fly as the keys are being logged to frustrate attempts and see what is going on. But the techniques for finding encrypted text will be similar to finding plaintext, even if the analysis is necessarily different¹².

Difficulties

The key logger had a number of deficiencies which were not addressed because the goal was to demonstrate the feasibility of extracting the memory address pointer and not developing an effective attack vector. The most obvious of these being the social engineering required to get a user to install an unsigned driver. Some less obvious ones were:

- The PFN Database moves around a good deal from version to version and some means would need to be developed to properly locate this. Failure to do so results in an immediate blue screen as soon as the driver attempts to update the database. This was encountered when the test machine applied a number of windows updates.
- The code used to briefly hook the keyboard in order to locate the keyboard buffer was set to occur when the driver was initially loaded. When the computer was subsequently rebooted this driver would load before the keyboard was available causing a blue screen on boot up. This could be remedied in several ways:
 - It may be possible to set the driver to load after everything else has loaded so that the keyboard would be more likely to be present.
 - The code that did the keyboard hooking/buffer discovery could be triggered from a command sent from the CUDA program prior to its requesting the pointer offsets. This would seem to be a more promising solution as it would allow the attacker more flexibility of when to begin looking.

Performance

There was no noticeable degradation of performance while running these tests. No attempt was made to measure any that may have been detectable but not noticeable.

¹² The central weakness addressed is the inability to examine GPU memory from the CPU.

Detection

There are currently no tools for scanning and examining video card memory in MS Windows which makes detecting this sort of key logger particularly difficult¹³. Most of the time, however, video memory persists through computer reboots¹⁴, which means that a Linux partition or a bootable Linux CD with the appropriate memory tools installed would be able to detect the existence of this rootkit.

NVIDIA Memory Dump

NVIDIA has released a (64-bit Linux only at present) patch which provides access to video memory¹⁵. While this seems promising, it could produce other security problems, especially in virtualized environments where it might be exploited to allow for data leakage if it were incorporated in the primary video card drivers. Since video memory is usually persistent through computer reboots, this looks to hold more promise as a forensic toolset where a PC with a suspected rootkit could be rebooted to use a forensic Linux distribution with the patched driver and the memory examined.

Exploring this on a dual boot (Fedora Linux/Windows 8.1) 64-bit laptop (an ASUS R751JB-MH71 with an NVIDIA GeForce GT 740M) revealed some peculiar behavior with respect to the video memory. Upon initial power on, there would be random data revealed in the video memory – which is unsurprising. What was surprising is that when rebooting the laptop, the video memory would be completely empty as if it had been wiped as part of the reboot process. This finding eliminated rebooting into a forensic Linux distribution as a possible means of reading GPU memory.

PCI Address Space

Maurice et al have demonstrated a way to access the video memory by physical address via the PCI address space¹⁶. There is a register called HOST_MEM at address 0x1700 which controls a 1MB window into the video memory which can be accessed in the physical address range of 0x700000-0x7FFFFFFF. This was replicated and incorporated into a Kernel Mode driver which could be called by a User Mode program to allow viewing and analysis.

PCI Video Memory Dump Driver

The basic flow of the PCI Video Memory Dump Driver is to obtain the physical memory address of BAR0 from the video card, calculate the location of the HOST_MEM and PRAMIN Window addresses. The driver then waits for a client program to call it with an offset value. Upon receipt of such a value, it writes that value to HOST_MEM and then copies the PRAMIN window to a supplied buffer to send back to the calling program.

13 [Ladakis] p 4

14 [Maurice] p 10

15 [Richard]

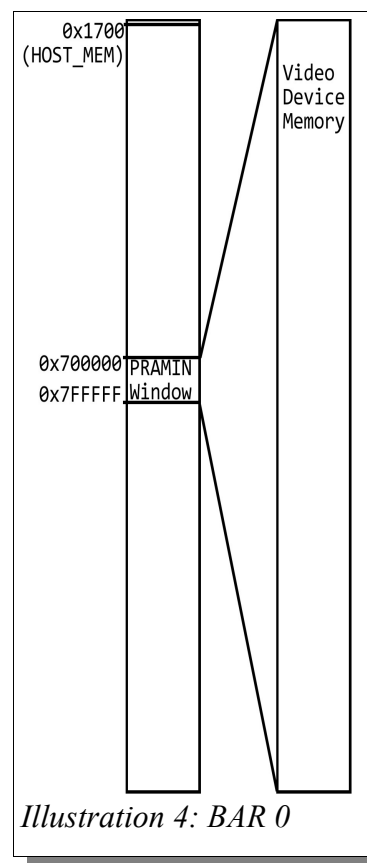
16 [Maurice] p 12-14

BAR 0

Bar 0, all the bars in fact, are set by the BIOS at boot time, and it is a range of physical memory addresses which the hardware maps to the video card¹⁷. This address of the \Driver\pci driver object was obtained by calling the same unpublished API call, `ObReferenceObjectByName`, which had been previously used to obtain the `kbdclass` and `kbdhid` driver objects previously. From there all the attached devices are iterated through until a video device (`DeviceType = FILE_DEVICE_VIDEO` or `0x23`) is obtained. The PCI Common Header is read from the device to get the first Base Address. This is the physical address for BAR0. The `HOST_MEM` address is `BAR0 + 0x1700` and the `PRAMIN_Window` address is `BAR0 + 0x700000`¹⁸.

Control Device

A Control Device is then set up to listen on \Dos Devices\DeviceMemDump for the Dump Client which supplies it with a window offset and a 1 MiB buffer. The Control Device first makes a note of the value currently in `HOST_MEM` before overwriting it with the DumpClient supplied offset value. Then the `PRAMIN_Window` memory is copied over to the supplied buffer. Once this is achieved it double-checks `HOST_MEM` to ensure nothing has changed the `PRAMIN` window while it was doing the copy. The buffer is finally returned back to the calling Dump Client and `HOST_MEM` is restored to its previous value.



Dump Client

The Dump Client repeatedly calls the Control Device with offset values beginning at 0 and incrementing by `0x10` until it has pulled down the entire 1024 MB. Each call it writes the pulled data out to a file. The client accepts the following arguments:

- **file** [*filename*] – writes the output to a file with the given file name.
- **pause** [*seconds*] – causes the client to pause for the given number of seconds between each kernel call for more memory.
- **full** – the entire memory will be requested in a series of multiple driver calls.
- [*hex address*] – a hex address can be entered, beginning with '0x' which will result in a single page being requested from the driver, beginning with the offset specified.

Proof of Concept

A series of proof of concept tests were run with a simple CUDA program that would take several long strings of text and copy them into the memory space of the video card. A second command shell was opened up to do memory dumps under the following conditions:

- First dump was done with the CUDA program still running.
- Second dump was done after exiting the program.

¹⁷ [Yao] p 16-17

¹⁸ [Maurice] p 12

- Third dump was done after closing the command shell under which the CUDA program had been running.

Analysis of the dumps was done by running a search of all strings over 10 characters long and dumping that to a secondary file which was opened in a text editor. All strings of data were repeatedly found under each of the above scenarios.

Performance

A serious difficulty with this technique is that while the memory dump was running, the test PC was slowed down to a crawl. This could have been due to:

- The memory copy operation – which would pose a serious obstacle to using this technique as this operation is always needed for this type of analysis.
- The client program writing a 1 GiB to disk – this would be a less serious problem as a real-world defense would involve real-time analysis and not the dumping of the entire video memory to disk.

To test this performance benchmarks were taken while running the Client Dump program under 4 different configurations and the impact on the test machine was measured.

NovaBench

NovaBench¹⁹ was chosen for measuring the impact of the several different memory scanning strategies because it was free, fairly simple to use, and provided the details of its tests and not simply an overall numeric score. I was interested in measuring the effects on system memory, the CPU, and the GPU. The 5 tests from NovaBench to measure these things were:

- **RAM Speed** – Measures the speed at which data can be written to and read from RAM.
- **Floating Point Test** – measures the number of operations per second at which the CPU can perform floating point arithmetic.
- **Integer Test** – measures the number of operations per second at which the CPU can perform integer arithmetic.
- **MD5 Hashing Speed** – measures the number of MD5 hashes the CPU can perform per second.
- **3D Graphics Test** – measures the number of frames per second at which the GPU can render a heavily shader dependent scene.

Baseline Test

First a baseline test was run with no memory dump activities executing. The results provide the goal for which any strategy needs to achieve in order to provide some degree of unobtrusive protection.

RAM Speed	5,575 MB/Second
CPU Tests	60,384,216 Floating Point Operations/Second
	150,253,162 Integer Operations/Second
	906,191 Hashes/Second
GPU Test	727 Frames/Second

Table 1: Baseline Results

No Pause Write

Running the dump program to dump the entire GPU device memory to a file on disk took an average of 5 minutes 51 seconds. This process execution caused the entire system to slow down to the

¹⁹ [NovaBench]

point where it impacted even moving the mouse around the screen. Running the NovaBench tool while this dump was occurring demonstrated the extent to which the memory dump degraded the performance of the PC.

RAM Test	3,526 MB/Second
CPU Tests	4,967,279 Floating Point Operations/Second
	16,791,938 Integer Operations/Second
	67,803 Hashes/Second
GPU Test	64 Frames/Second

Table 2: No Pause Write Results

No Pause No Write

Running the dump program to grab the memory from the video card but to not have it dump the video to a file took 5 minutes 44 seconds to do. Just as in the first test there was a significant impact in my impression of how responsive the system became. This was born out by the results from the

RAM Test	2,137 MB/Second
CPU Tests	1,790,917 Floating Point Operations/Second
	5,915,372 Integer Operations/Second
	38,479 Hashes/Second
GPU Test	32 Frames/Second

Table 3: No Pause No Write Results

NovaBench analysis during this dump which revealed that, if anything, the PC's performance was even much worse than when the memory file was not being written to disk.

10 Second Pause

Adding a 10 second pause between each kernel call and writing the contents to disk resulted in no appreciable degradation in system responsiveness when compared to the original baseline. It extended the time the memory scan required to complete from the previous times of less than 6 minutes to 2 hours, 50 minutes, and 30 seconds. The number of frames rendered per second actually went up by a small amount when compared against the baseline (though obviously this indicates that there is some amount of variation in the operating system and/or other background processes as it is absurd to attribute a performance improvement to this memory scanning process.)

RAM Test	5,022 MB/Second
CPU Tests	51,517,702 Floating Point Operations/Second
	145,357,310 Integer Operations/Second
	942,713 Hashes/Second
GPU Test	826 Frames/Second

Table 4: 10 Second Pause Results

60 Second Pause

The final test was to increase the pause between each kernel call to fetch memory to 60 seconds. Writing the output to disk was not re-introduced as it was unlikely to provide interesting results and a real memory scanning tool is unlikely to want to dump the memory to disk. There was no significant improvement in my own subjective experience of the system and this was reflected in the benchmark results showing some modest improvements in some tests and small degradation in others. The time

RAM Test	4,636 MB/Second
CPU Tests	60,765,344 Floating Point Operations/Second
	128,194,788 Integer Operations/Second
	943,973 Hashes/Second
GPU Test	856 Frames/Second

Table 5: 60 Second Pause Results

required for this scan to complete was increased to almost 18 hours, with little appreciable difference to the 10 second pause.

Performance Conclusions

The resource drain caused by this process appears to be directly related to the extraction of memory from the video card and completely unrelated to the I/O of writing the data to an output file for analysis. In fact, writing the file even seemed to have a positive effect on PC performance (assuming this is due to the likely slowing down of the client program necessitated by the file IO.) Given the large performance impact that the memory copy operation seems to have, it seems reasonable to expect that this technique for viewing video memory is probably best done a single page at a time every few minutes.

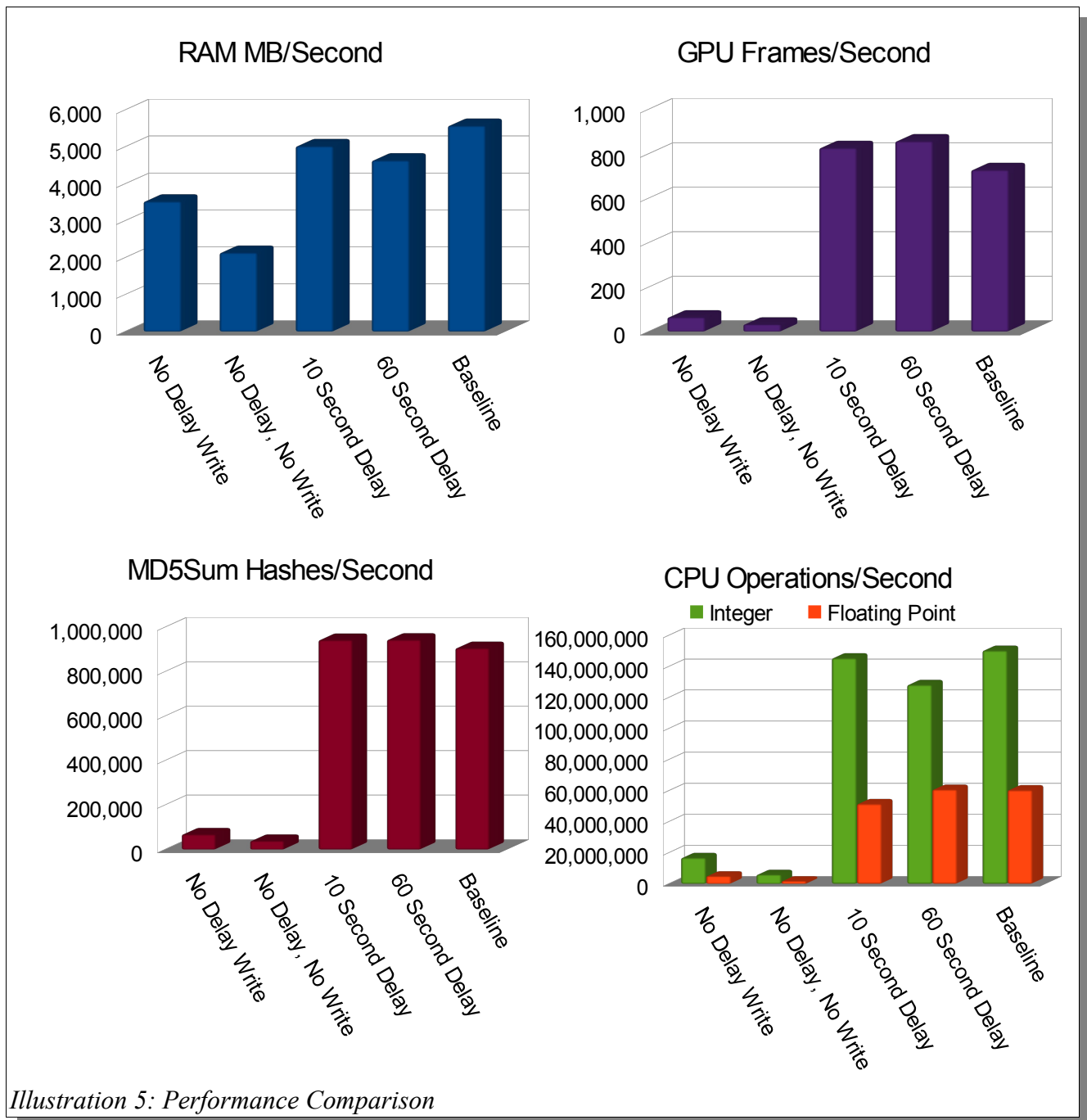


Illustration 5: Performance Comparison

Speculation and Future Work

The exploit in this project focused on keystroke logging. Keystroke buffers are often rich targets for attackers due to the sensitive data (like usernames, passwords, credit cards, banking information, etc...) that is often typed and therefore exposed by these loggers. At some point this may become less valuable if other identification/authentication systems (CAC cards, fingerprint readers, SQRL, etc...) proliferate. The techniques used here, however, are not limited in application to attaching the keyboard buffer – they can be easily applied to any memory accessible to the Windows Kernel (including CAC cards, fingerprint readers, etc...).

The base technique used here is to pass a memory pointer out of the kernel to a CUDA program. The passing of this pointer only has to happen once, and from that point on the program's execution and manipulation of the memory address happens out of sight of the CPU. It might read from or write to the address.

Monitoring memory usage via the PCI BUS appears to have some potential for future work, but this involves directly mapping a physical memory address and a physical memory address range, both of which may be simultaneously used for other purposes. It would be better for this function to be provided by the official video card drivers so that there would only be a single process managing this space. Alternatively an additional hardware range could be supplied to provide read-only access to the memory for use by anti-malware software drivers.

Illustrations

Illustration 1: Typical X86 Architecture.....3

Illustration 2: Device Tree.....5

Illustration 3: Un-Named Device.....6

Illustration 4: BAR 0.....11

Illustration 5: Performance Comparison.....14

Tables

Table 1: Baseline Results.....12

Table 2: No Pause Write Results.....13

Table 3: No Pause No Write Results.....13

Table 4: 10 Second Pause Results.....13

Table 5: 60 Second Pause Results.....13

References

- Wilt: Nicholas Wilt, The CUDA Handbook: A Comprehensive Guide to GPU Programming, 2013
- Stewin: Stewin, Patrick, Detecting Peripheral-based Attacks on the Host Memory, 2015
- Vasiliadis: Giorgos Vasiliadis, Michalis Polychronakis, Sotiris Ioannidis , GPU-Assisted Malware, 2015
- Wikipedia: Wikipedia contributors, GeForce 6 series, 22 May. 2015,
http://en.wikipedia.org/w/index.php?title=GeForce_6_series&oldid=663565206
- freedesktop: , "HwIntroduction", 27 May 2015, <http://people.freedesktop.org/~mslusarz/nouveau-wiki-dump/HwIntroduction.html>
- Rossovich: Rossovich, Arkady, The eternal kbfiltr question, 2009,
<https://www.osronline.com/showthread.cfm?link=166463>
- Russinovich: Mark Russinovich, David A. Solomon, Windows® Internals: Including Windows Server 2008 and Windows Vista, 2009
- Chen: Chen, Raymond, Some remarks on VirtualAlloc and MEM_LARGE_PAGES, 2011,
<http://blogs.msdn.com/b/oldnewthing/archive/2011/01/28/10121300.aspx>
- Ladakis: Evangelos Ladakis, Lazaros Koromilas, Giorgos Vasiliadis, Michalis Polychronakis, Sotiris Ioannidis, You Can Type, but You Can't Hide: A StealthyGPU-based Keylogger, 2013
- Richard: Dr. Golden Richard III, 2015 DFRWS Forensics Challenge Released!, 2015,
<http://cs.uno.edu/~golden/gpu-malware-research.html>
- Maurice: Clementine Maurice, Christoph Neumann, Olivier Heen, and Aurelien Francillon, Confidentiality Issues on a GPU in a Virtualized Environment, 2014
- Yao: Jiewen Yao, Vincent J. Zimmer, A Tour beyond BIOS Memory Map Design in UEFI BIOS, 2015,
https://firmware.intel.com/sites/default/files/resources/A_Tour_Beyond_BIOS_Memory_Map_in%20UEFI_BIOS.pdf
- NovaBench: , About NovaBench, , <https://novabench.com/about.php>